xUnit Assertion Methods

Assertions are the **verification mechanism** in **xUnit.net**. They confirm whether the *actual outcome* of a unit under test matches the *expected behavior*.

Below is a **structured, method-by-method explanation** using simple, realistic examples.

---

1. Sample Production Code (Multiple Methods)

```
public class Calculator
{
    public int Add(int a, int b) => a + b;

    public int Divide(int a, int b)
    {
        if (b == 0)
            throw new DivideByZeroException();

        return a / b;
    }

    public bool IsEven(int number) => number % 2 == 0;

    public string? GetUserName(int userId)
    {
        return userId == 1 ? "Admin" : null;
    }

    public List<int> GetNumbers()
```

```
    {
        return new List<int> { 1, 2, 3 };
    }
}
```

---

2. Assert.Equal() and Assert.NotEqual()

Purpose

Validate **expected vs actual values**.

```
[Fact]
public void Add_TwoNumbers_ReturnsCorrectSum()
{
    var calculator = new Calculator();

    var result = calculator.Add(10, 20);

    Assert.Equal(30, result);
    Assert.NotEqual(40, result);
}
```

Use cases:

- Calculations
- Business rule outputs
- DTO mapping values

---

3. Assert.True() and Assert.False()

Purpose

Validate **boolean conditions**.

```
[Theory]
[InlineData(2)]
[InlineData(4)]
public void IsEven_EvenNumber_ReturnsTrue(int number)
{
    var calculator = new Calculator();

    var result = calculator.IsEven(number);

    Assert.True(result);
}

[Fact]
public void IsEven_OddNumber_ReturnsFalse()
{
    var calculator = new Calculator();

    var result = calculator.IsEven(3);

    Assert.False(result);
}
```

Use cases:

- Flags
- Validation rules
- Conditional logic

---

4. Assert.Null() and Assert.NotNull()

Purpose

Verify **nullability behavior**.

```
[Fact]
public void GetUserName_InvalidUser_ReturnsNull()
{
    var calculator = new Calculator();

    var result = calculator.GetUserName(99);

    Assert.Null(result);
}

[Fact]
public void GetUserName_ValidUser_ReturnsName()
{
    var calculator = new Calculator();

    var result = calculator.GetUserName(1);

    Assert.NotNull(result);
}
```

Use cases:

- Repository results
- Optional data
- Defensive programming

---

5. Assert.Throws<T>() (Exception Testing)

Purpose

Verify **expected exceptions**.

```
[Fact]
public void Divide_ByZero_ThrowsException()
{
    var calculator = new Calculator();

    Assert.Throws<DivideByZeroException>(() =>
        calculator.Divide(10, 0));
}
```

Key point:

- The code **must be wrapped in a lambda**

---

6. Assert.ThrowsAsync<T>() (Async Exception Testing)

```
public async Task<int> DivideAsync(int a, int b)
{
    if (b == 0)
        throw new DivideByZeroException();

    return await Task.FromResult(a / b);
}
[Fact]
public async Task DivideAsync_ByZero_ThrowsException()
{
```

```
    var calculator = new Calculator();


    await Assert.ThrowsAsync<DivideByZeroException>(() =>
        calculator.DivideAsync(10, 0));
}
```

Use cases:

- Async service methods
- API calls
- Background processing

---

7. Assert.Contains() and Assert.DoesNotContain()

Purpose

Validate **collections or strings**.

```
[Fact]
public void GetNumbers_ShouldContainValue()
{
    var calculator = new Calculator();


    var result = calculator.GetNumbers();


    Assert.Contains(2, result);
    Assert.DoesNotContain(5, result);
}
```

---

8. Assert.Single() and Assert.Empty()

```
[Fact]
```

```
public void Collection_ShouldHaveSingleItem()
{
    var list = new List<int> { 10 };

    Assert.Single(list);
}

[Fact]
public void Collection_ShouldBeEmpty()
{
    var list = new List<int>();

    Assert.Empty(list);
}
```

Use cases:

- Query results
- Filtered collections
- Repository responses

---

9. Assert.IsType<T>() and Assert.IsNotType<T>()

Purpose

Validate **object types**.

```
[Fact]
public void Result_ShouldBeOfExpectedType()
{
    object value = 10;
```

```
    Assert.IsType<int>(value);
    Assert.IsNotType<string>(value);
}
```

---

10. Assert.InRange() (Boundary Testing)

```
[Theory]
[InlineData(10)]
[InlineData(50)]
public void Number_ShouldBeWithinRange(int number)
{
    Assert.InRange(number, 1, 100);
}
```

Use cases:

- Age validation
- Limits and thresholds
- Score calculations

---

11. Summary of Common Assertions

Assertion	**Purpose**
Equal / NotEqual	Value comparison
True / False	Boolean validation
Null / NotNull	Null checks
Throws / ThrowsAsync	Exception validation

Assertion	Purpose
Contains	Collection/string check
Single / Empty	Collection size
IsType	Type checking
InRange	Boundary validation

---

12. Best Practices

- Prefer **specific assertions** over generic ones
- One logical assertion group per test
- Always assert **behavior**, not implementation
- Use exception assertions for **negative scenarios**
- Combine [Theory] with assertions for broad coverage